SpatialTwin-IoT: A Bidirectional Mixed-Reality and Web3D Digital-Twin Framework for Smart Fluid Systems and Interactive Industrial Assemblies

Gopal Krishna J S, M Aaryamann Prabhu, Dr. Andhe Dharani
Department of Master of Computer Applications
RV College of Engineering®, Bengaluru, India

***Abstract*—Low-cost edge sensors make it increasingly practical to monitor plumbing networks, valve assemblies, and small industrial fluid systems. However, inspection and configuration are often split between browser-based tools and immersive mixed-reality (MR) applications, which may not maintain a shared, authoritative representation of the same physical assembly. This paper presents SpatialTwin-IoT, a proposed bidirectional digital-twin framework that brings together an ESP32 telemetry tier, a FastAPI synchronization service with a versioned WebSocket project channel, a browser-based Web3D configurator implemented with React Three Fiber, and MR pathways for Microsoft HoloLens 2 and XREAL One Pro. The design extends the existing PipeLab plumbing-network editor and is organized around four contributions. First, it defines a device-independent plumbing-network model that links sensors and viewers through stable component and port identifiers. Second, it specifies a synchronization scheme in which sensor measurements and client edits carry explicit origin, timestamps, revisions, and authoritative-state semantics, with separate ownership rules for telemetry and configuration fields. Third, it defines an engineering evaluation plan with pre-registered targets of less than 100 ms for sensor-to-client latency and 60 frames per second for Web3D rendering on named reference hardware. Fourth, it identifies the portability boundary between shared data structures and device-specific tracking, anchoring, input, and rendering features. The work distinguishes the existing PipeLab baseline from the planned integrations and treats the stated performance values as targets to be tested, not as achieved results.**

***Index Terms*—Augmented reality, digital twin, mixed reality, Internet of Things, spatial registration, Web3D.**

I. Introduction

Industrial fluid systems range from building plumbing to compact process lines. When faults propagate through valves, pumps, pipes, and outlets, locating the source can become time-consuming. Low-cost microcontrollers such as the Espressif ESP32 make it possible to collect flow, pressure, and level measurements from these assemblies. At the same time, mixed-reality headsets such as Microsoft HoloLens 2 and XREAL One Pro, together with browser-based 3D renderers such as Three.js and React Three Fiber, provide ways to inspect system state in spatial or web-based interfaces.

The difficulty is not simply displaying sensor data. The physical sensors, browser configurator, and immersive viewer must agree on which components exist and what state each component holds. A valve edit made in a browser, for example, should be reflected in the MR view; a new flow measurement should appear consistently in all connected clients. If each viewer maintains its own state, a user can end up inspecting a representation that no longer matches the authoritative project state. A useful digital twin therefore needs a defined source of truth, explicit update semantics, and measurable limits on the time required to propagate changes.

Earlier work has explored MR interfaces for digital twins, communication between simulated and physical systems, and browser-delivered twin platforms [1], [2], [4], [7]–[12]. These studies provide a foundation for immersive inspection and web-based collaboration. However, the combination considered here—an ESP32-class sensing tier, an MR viewer, and a browser-based Web3D configurator—requires a shared synchronization model that records the origin, timestamp, and revision of updates. Reproducibility also depends on documenting what each headset can actually support, rather than assuming that tracking and anchoring capabilities are equivalent across devices.

This paper develops SpatialTwin-IoT as an extension of PipeLab, an existing browser-based plumbing-network editor. The proposed extension combines a typed component-and-port graph, stable identifiers, a FastAPI service, project persistence, and a versioned WebSocket event channel with edge telemetry and MR clients. Its contributions are bounded and intended to be verifiable:

1) A device-independent plumbing-network model that connects an ESP32 telemetry path, an MR viewer, and a Web3D configurator through shared component and port identifiers.
2) A bidirectional synchronization design that records message origin, timestamps, revisions, and authoritative-state semantics, while separating telemetry-owned fields from user-owned configuration fields.
3) An evaluation methodology with pre-registered latency and rendering targets, along with defined measurement boundaries and reference hardware requirements.
4) A description of the portability boundary between the shared layer—identifiers, graph structure, units, and events—and device-specific features such as tracking, anchors, input, and rendering.

The distinction between the existing baseline and the proposed extension is important. The current PipeLab baseline

provides the Web3D scene, typed graph, API, persistence, and project-event channel. ESP32 integration, live telemetry, MR clients, spatial-anchor support, device studies, and performance measurements are described as planned work rather than completed functionality. The paper therefore presents an architecture and evaluation plan; it does not claim that the stated performance targets have already been met.

II. Related Work

A. Mixed-Reality Digital Twins

Tu *et al.* presented an MR interface for a digital-twin-based overhead crane, demonstrating how holographic views of live equipment state can support operator awareness [1]. Their later work, TwinXR, uses ontology-based digital-twin descriptions in industrial extended-reality applications and describes a Unity package for exchanging data between XR applications and server-side twin descriptions [2]. Hao *et al.* developed an MR digital twin for monitoring a roadheader in an underground mining context [4]. Kaarlela *et al.* investigated the use of XR-based digital twins in robotic training [7], while Tran *et al.* described an immersive augmented-reality interface for large USD-based digital twins [8]. Collectively, these studies show how immersive interfaces can connect digital representations with physical assets. Their emphasis is largely on visualization, interaction, or training rather than on a shared state model spanning low-cost edge sensing, MR, and Web3D clients.

B. Bidirectional Synchronization and Real-Time Protocols

Bidirectional communication between simulations and physical systems has been examined in robotics and construction. The ISARC consortium reported a communication bridge for synchronizing digital-twin simulations with physical construction robots and evaluated pose synchronization between robots [10]. Yenduri *et al.* examined latency in WebSocket and industrial protocols for digital-twin integration, providing context for protocol selection and message sizing [9]. Li *et al.* proposed a digital-twin platform for building performance and fluid monitoring using open web standards [11]. Lei *et al.* discussed implementation approaches for web-based digital-twin laboratories and remote state propagation [12]. These studies inform the decision to begin with the existing FastAPI service and WebSocket channel. A separate broker, such as Redis, is treated as a scaling option to be justified by measured fan-out limits rather than as an assumed requirement.

C. Cross-Reality Collaboration and Web3D

Lee and Yoo discussed XR collaboration that extends beyond fully virtual environments into work performed in the physical world [5]. Vemuri *et al.* described cloud-based collaborative VR laboratories using web-simulated microarchitectures, illustrating the practicality of delivering interactive 3D scenes through browsers [6]. The Web3D pathway in SpatialTwin-IoT builds on this approach. Because PipeLab already has a React Three Fiber scene, a typed component-and-port graph, and a WebSocket connection for project events, the proposed extension can reuse the rendering foundation instead of replacing it.

D. On-Device Interaction and Resource Constraints

Valladares-Poncela *et al.* studied on-device speech recognition and interaction for low-resource languages in industrial MR applications [3]. This work provides context for treating audio as an optional interaction mode rather than a dependency of the initial system. Spatial audio should only be mapped to measured flow after the sensors have been calibrated and the shared-state workflow has been validated. Similarly, speech interaction is left disabled by default until the baseline inspection and configuration tasks work reliably.

E. Research Gap

The cited work does not, as far as the literature considered here establishes, bring together all four elements targeted by this paper: (i) an ESP32-class edge tier with documented pulse-counting and analog-acquisition behavior; (ii) a bidirectional state model that distinguishes telemetry-owned from user-owned fields; (iii) synchronized MR and Web3D clients; and (iv) a cross-device evaluation that records the actual tracking capabilities of HoloLens 2 and XREAL One Pro rather than emulating equivalent capabilities. SpatialTwin-IoT addresses this combination at the architectural and evaluation-design level. Its claims remain limited to the hardware, implementation work, and measurements that can be documented when the evaluation is completed.

III. System Requirements and Architecture

A. Use Case: Smart Fluid Assembly

The reference use case is a compact physical assembly consisting of a tank, a pipe segment, a manual or solenoid-actuated valve, an ESP32-connected flow sensor, and an outlet. Four operator tasks define the intended workflow: (1) locate a named valve and identify its upstream and downstream connections; (2) locate a reported warning or failure and identify its stated cause and affected outlet; (3) change a valve state or select a specified component, then confirm that a second client or refreshed session shows the same state; and (4) realign or recover the network after a controlled tracking interruption, but only on a device and configuration that support a fair test of that task.

These tasks are defined in terms of the underlying assembly state rather than a particular display. This makes a cross-client evaluation possible once the shared state and event behavior have been implemented and verified.

B. Design Principles

The architecture follows four principles.

One authoritative state: The service maintains the authoritative project state. Clients retrieve a snapshot, represent entities using stable component identifiers, subscribe to project events, and submit edits through the API. A client does not become an independent source of truth.

Explicit origin and timestamps: Each telemetry sample and client edit carries an origin identifier, a capture or client timestamp, a service receive timestamp, and a sequence number

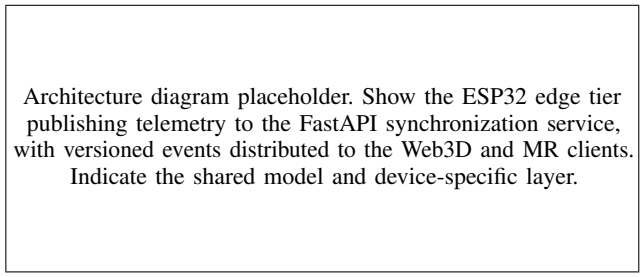


Fig. 1. Three-tier architecture of SpatialTwin-IoT. The ESP32 edge tier publishes timestamped telemetry to the FastAPI synchronization tier, which maintains the authoritative project state and broadcasts versioned events to Web3D and MR clients. The field-ownership rules distinguish sensor-owned measurements from user-owned configuration.

or revision. These fields allow event ordering and latency boundaries to be examined from logs.

Field ownership: Measurements such as flow rate and pulse count are telemetry-owned, while valve setpoints, component placement, and configuration toggles are user-owned. Sensor messages must not silently overwrite configuration fields, and client edits must not silently replace sensor measurements.

Shared model, device-specific interaction: Component and port identifiers, graph structure, units, event envelopes, and API semantics are shared. Tracking, spatial anchors, input methods, and rendering remain specific to each client pathway and are documented separately.

C. Three-Tier Architecture

Fig. III-C summarizes the proposed architecture. In the edge tier, an ESP32 publishes timestamped and sequence-numbered measurements to a validated telemetry endpoint. The synchronization tier extends the existing FastAPI service and versioned WebSocket project channel. It validates incoming messages, assigns service timestamps, applies sequence and revision rules, updates the authoritative twin state, persists or buffers changes as appropriate, and broadcasts versioned events. The client tier includes an MR client for in-context inspection and the existing React Three Fiber client for browser-based visualization and configuration.

A shared data layer defines projects, components, ports, connections, telemetry, and event schemas. A separate device layer represents camera pose, tracking state, anchoring, input, and rendering. This separation is intended to prevent a device-specific coordinate or interaction mechanism from changing the meaning of the underlying engineering model.

D. Coordinate and Anchoring Model

The coordinate model uses an explicit transform chain from component-local coordinates to project or world coordinates and then to the device's tracking space. Component geometry is stored in component-local coordinates. The transform between the project and the physical assembly, or a reference to a spatial anchor, is stored separately; it is not inferred from a headset's transient origin.

A calibration procedure should expose visible reference geometry and support realignment and reset. QR-based alignment is a candidate calibration method, while millimetre-level precision remains an unverified target. Any eventual accuracy claim should be based on translational and rotational error measured against a surveyed reference across repeated alignments and relocalization events. Anchor creation, restoration, and tracking-loss events should be logged for subsequent analysis.

E. Data Ownership and Conflict Handling

The synchronization model separates fields into two ownership classes. Telemetry-owned fields include flow rate, pulse count, analog-to-digital-converter (ADC) readings, and sensor-quality flags. Only sensor-originated messages may write these fields; clients treat them as read-only. User-owned fields include valve setpoints, component placement, and configuration toggles. These fields are written through client-originated edits and must not be overwritten by telemetry.

Each update carries a revision identifier. The service rejects stale revisions and emits a divergence event when a client's local revision falls behind the authoritative revision. A sensor update and a configuration edit may arrive close together, but the ownership rules prevent them from silently writing the same field. This does not eliminate all synchronization failures: missing messages, stale clients, or invalid transitions still need to be recorded and handled explicitly.

IV. Implementation Design

A. PipeLab Baseline

PipeLab is the starting point for the proposed system. Its front end renders a React Three Fiber scene backed by a typed component-and-port graph. Components and connections have stable identifiers intended to be reused by all viewers, while ports store local positions and orientations. The API produces simulation results rather than relying on separate calculations in each client. The back end is a FastAPI service with project persistence and a WebSocket project channel that distributes events to connected clients.

The existing simulator is simplified and deterministic. It is not a validated hydraulic solver, computational-fluid-dynamics (CFD) model, or water-hammer model. Its outputs must therefore be described as visualization or test data unless they are replaced by, and validated against, an appropriate engineering reference. At the time described by this paper, the repository does not yet contain ESP32 integration, live telemetry, an MR client, spatial-anchor implementation, a device study, or measured evaluation results. Those items are planned extensions, not capabilities of the existing baseline.

B. Edge Tier: ESP32

The planned ESP32 pathway counts flow-sensor pulses, using an interrupt service routine where appropriate, and acquires analog values at the resolution supported by the selected board. Measurements must be calibrated against a reference. Each telemetry message should include the sampling interval, units (for example, litres per minute), quality flags, device identity, capture timestamp, and sequence number. Quality flags may identify a saturated sensor, degraded signal, or stale calibration.

Bounded queues and explicit reconnect behavior are intended to prevent a slow network connection from blocking acquisition indefinitely. The design does not assume that every pulse will be counted. Interrupt-service-routine behavior, ADC noise, and physical installation can introduce bias or missed events. Bench tests should compare pulse counts with a controlled pulse source across target flow rates and report the observed loss. Soil-moisture sensing is optional and should be included only if it has a defined role in the smart-fluid assembly evaluation.

C. Synchronization Tier: Service and Broker

The synchronization tier extends the existing FastAPI service and WebSocket project channel. Telemetry and command schemas are versioned. If the service is exposed beyond a trusted laboratory network, an authentication policy is required. On receiving a message, the service validates its contents, assigns a receive timestamp, applies sequence and revision rules, updates the twin state, persists or buffers the change where required, and broadcasts a versioned event.

JSON is appropriate for the initial prototype. Payload size, message rate, and fan-out behavior should be measured before optimizing the transport. Redis or a dedicated broker should be introduced only if benchmarking shows that the single-process design has become a limiting factor. A separate Node.js relay remains a possible alternative, not a current dependency; running multiple relays without clear responsibility boundaries would complicate the system.

On reconnect, a client receives a snapshot of the current authoritative state and then any missed events available within a defined replay window. The snapshot-and-replay procedure should be tested to determine whether a reconnecting client reaches the same state as a continuously connected client. The implementation must also define what happens when the replay window is insufficient or the client's schema version is no longer supported.

D. Web3D Client

The Web3D pathway extends the existing PipeLab browser editor into a stakeholder configurator. The component graph is kept separate from Three.js render objects. Selection state, configuration toggles, and exploded-view offsets should have explicit view or configuration semantics rather than being encoded as unintended changes to engineering geometry or connections.

Additional features—including physically based rendering (PBR) material toggles, exploded views, Draco-compressed imported meshes, animated fluid effects, and audio—should be introduced only after the shared state and baseline tasks work end to end. Draco compression should be benchmarked on imported mesh payloads by reporting transfer size, load time, and frame rate. Asset optimization and rendering enhancements should follow baseline profiling rather than precede it.

E. MR Client: HoloLens 2 Pathway

The HoloLens 2 pathway is planned as a Unity/OpenXR client, subject to the development environment and runtime features available on the actual device. Hand interaction and spatial anchors are the main proposed interaction and registration mechanisms, with gaze used for targeting where appropriate. Required runtime features must be verified against the installed operating system, runtime, and toolchain; support should not be inferred solely from documentation of an OpenXR extension.

Spatial-mapping and persistent-anchor events should be logged, including anchor creation, restoration, and tracking loss, so that drift and recovery can be evaluated offline. HoloLens 2 provides native six-degrees-of-freedom (6DoF) world tracking, which provides the basis for evaluating physical-assembly alignment on this pathway.

F. MR Client: XREAL One Pro Pathway

The XREAL One Pro pathway depends on the available host device, SDK/runtime, and operating mode. XREAL describes built-in three-degrees-of-freedom (3DoF) spatial modes powered by its X1 spatial chip. Built-in rotational tracking does not, by itself, provide headset-native 6DoF world tracking, spatial mapping, hand tracking, or persistent world anchors. The exact capabilities available in the selected host and accessory configuration must be checked during implementation.

If stable, room-locked placement requires a supported host-tracked or accessory-based setup, that configuration should be identified and documented. Otherwise, the XREAL condition should use a screen-anchored, head-following, or explicitly host-tracked workflow. This difference is part of the device comparison and must not be hidden by treating the two headsets as equivalent.

G. Message and Event Schemas

The telemetry schema includes a device identifier, sensor identifier, capture timestamp, sequence number, units, measurement value, and quality flags. Timestamps are expressed in milliseconds since the epoch when sourced from a synchronized clock. If an edge-side monotonic clock is used, its offset and relationship to the service clock must be documented.

A command message includes the client identifier, target component or port identifier, field name, value, and client-side revision token. The field name must be restricted to user-owned fields. The event envelope records the originating message identifier, service-assigned revision, broadcast timestamp, and schema version. Clients are expected to acknowledge receipt within a configurable interval. Missing acknowledgements should trigger a divergence event, and clients advertising an unsupported schema version should be rejected at connection time or guided through an explicitly defined compatibility path.

V. Evaluation Method

A. Staged Evaluation

Evaluation is planned in stages. Engineering bench experiments come first and cover telemetry accuracy, end-to-end latency, state convergence, reconnect behavior, and browser rendering performance. A small user study may follow if time,

TABLE I
INTERACTION AND TRACKING ACROSS THE THREE CLIENT PATHWAYS

Tracking configuration	Primary input	Anchoring / coordinate behavior
6DoF world-locked (HoloLens 2 pathway)	Hand interaction and gaze	Native spatial anchors, subject to runtime verification
Built-in 3DoF rotational tracking (XREAL One Pro pathway)	Depends on host and available input	Screen-anchored or host-tracked unless a supported configuration provides stronger registration
No headset tracking (2D Web3D viewport)	Mouse, keyboard, or touch	Project-coordinate representation

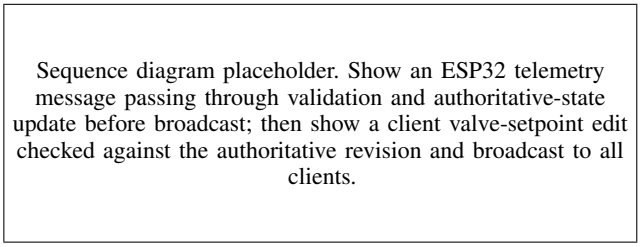


Fig. 2. Sequence of a bidirectional update cycle. Telemetry includes a capture timestamp and sequence number; the service validates and timestamps the message, updates the authoritative state, and broadcasts a revisioned event. Client edits are checked against the authoritative revision before the merged state is distributed.

hardware, and ethics approval permit. A within-subject comparison between MR devices is appropriate only when each device condition supports a clearly documented path through the task; participant order should then be counterbalanced. A pilot should establish task wording, training time, instrumentation, and a feasible sample size. The final sample size and rationale must be reported after study planning. No participant count should be presented as though data had already been collected.

If only one MR configuration is practical, the study should be framed as a feasibility evaluation and cross-device comparative claims should be removed.

B. Bench Setup

The bench setup should use a reference flow meter or controlled pulse source to validate ESP32 measurements. The network setup—wired or wireless—must be documented, including bandwidth, latency, and packet loss. The Web3D test machine should be identified by processor, memory, GPU, and browser version. Any MR device, host system, operating-system version, SDK, and runtime used in a test should also be reported.

All clock-based latency measurements must share a documented time reference, such as clocks synchronized through NTP with a measured offset. If this cannot be guaranteed, the measurement method should avoid subtracting unsynchronized wall-clock timestamps. For each telemetry sample, the logs should record sensor capture, service receipt, service broadcast, client receipt, and visible-render timestamps where available. Acknowledgements, sequence gaps, and command revisions should also be retained.

The latency boundaries must be defined before data collection. These include sensor capture to client-visible update, client edit to peer-visible update, server round trip, and motion-to-photon latency where credible instrumentation is possible. Motion-to-photon latency should be omitted if it cannot be measured reliably rather than estimated indirectly.

C. Measures

Table II summarizes candidate evaluation dimensions, measures, and collection methods. For latency, results should include median and tail percentiles such as the 95th percentile, together with packet loss, workload, client count, network conditions, and missing data. The targets of less than 100 ms for sensor-to-client latency and 60 frames per second for Web3D rendering are requirements to test. They are not results until the relevant measurements are collected on named hardware.

D. Tasks and Success Criteria

The same underlying project states and task instructions should be used across the Web3D pathway and each MR pathway. The four tasks defined in Section III-A should be used, with success criteria set before formal data collection. For the first task, success means identifying the correct valve and its upstream and downstream components. For the second, it means correctly reporting the stated cause and affected outlet. For the third, it means verifying the updated shared state from a second client or refreshed session within the configured acknowledgement window. The fourth task is evaluated only on a device configuration that supports fair realignment after a controlled tracking interruption.

For each task, record completion, errors, time, help requests, and whether the server-side state matches the intended result. Identical task instructions do not guarantee identical interaction costs, so device-specific input and setup differences must be included when interpreting the results.

E. Participants and Ethics

Where feasible, participants should be recruited from a population resembling the intended users, and prior AR/MR experience should be recorded. Institutional or departmental ethics approval, or a documented exemption, must be obtained before recruitment. The study should include informed consent and clear privacy and data-retention procedures. Gaze and eye-tracking data should be collected only when necessary, approved, and disclosed. Participants should be screened for comfort and safety concerns, offered breaks, and allowed to withdraw at any time without penalty. A small convenience

TABLE II
EVALUATION DIMENSIONS, CANDIDATE MEASURES, AND COLLECTION METHODS

Dimension	Candidate measures	Collection method
Edge sensing	Flow measurement error, pulse-count loss, sampling interval, out-of-order or missing samples	Reference meter, controlled pulse source, and ESP32 logs
Synchronization	Sensor-to-client and action-to-peer latency distributions, delivery rate, event recovery, state divergence	Timestamped service/client logs and revision records
Spatial registration	Translation and rotation error, drift, recovery success after tracking interruption	Surveyed reference fixture and event logs
Web3D rendering	Frame-rate distribution, asset transfer size, load time, selection and configuration response time	Browser performance instrumentation
Task performance	Completion rate and time, selection misses, edit errors, help requests	Observer checklist and application logs
Device resources	Frame rate, reconnect duration, and memory use where available	Device and application profiling
Usability and workload	System Usability Scale (SUS), NASA Task Load Index (NASA-TLX), short interview	Post-condition questionnaires and debrief
Comfort	Discomfort rating, reasons, and session duration	Repeated ratings and debrief

sample supports exploratory feasibility findings rather than population-wide generalization.

F. Analysis Plan

Participant characteristics and descriptive results should be reported first. Paired measurements should be analyzed using tests appropriate to the observed distributions and sample size. Effect sizes and confidence intervals should accompany p-values, and multiple-comparison corrections should be applied when several outcomes are tested. Qualitative responses should be coded using a documented method, with anonymized excerpts used where appropriate.

The analysis is organized around five research questions:

- **RQ1:** Do sensor-originated telemetry updates and client-originated edits converge to the same authoritative state across Web3D and MR clients during normal operation and after reconnects?
- **RQ2:** What end-to-end latency and event-delivery behavior does the prototype exhibit under specified sensor rates, network conditions, and client counts?
- **RQ3:** Which inspection and assembly tasks are feasible with the tracking and input configurations actually available on HoloLens 2 and XREAL One Pro?
- **RQ4:** What frame rate and asset-loading behavior does the browser configurator achieve on the named reference hardware?
- **RQ5:** Which tracking, input, rendering, and deployment constraints affect use of the framework across reality interfaces?

G. Pre-Registered Performance Targets

Table III lists the intended targets and their measurement boundaries. The hardware and boundary for each result must be specified before a target is reported as achieved.

VI. RESULTS REPORTING PLAN

The results section will be populated after the bench evaluation has been completed. The figures described below specify the intended presentation; they are placeholders and do not represent measured results.

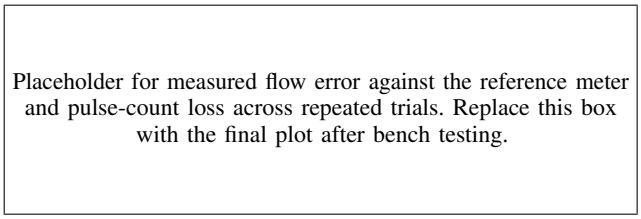


Fig. 3. Edge telemetry accuracy: (a) flow measurement error versus the reference meter at three target flow rates; (b) pulse-count loss across repeated trials. To be populated after bench evaluation.

A. Edge Sensing

The sensing evaluation should report the distribution of flow measurement error against a reference meter at the target flow rates, pulse-count loss percentage, sampling-interval distribution, and the number of missing or out-of-order samples per session. The reference meter model and controlled pulse-source configuration should be documented so that the results can be interpreted and reproduced.

B. Synchronization Latency

The synchronization evaluation should report median and 95th-percentile sensor-to-client latency, median and 95th-percentile action-to-peer latency, event delivery and recovery behavior during disconnect/reconnect, and state-divergence counts. Results should be stratified by sensor rate, network condition, and number of clients. Bandwidth, network latency, and packet loss should be recorded for each scenario.

C. Registration Accuracy

Registration results should include translational and rotational error against the surveyed reference, drift over time, and recovery success after a controlled tracking interruption. Results should be reported separately for the HoloLens 2 pathway and the supported XREAL One Pro configuration, using the same

TABLE III
PRE-REGISTERED PERFORMANCE TARGETS AND MEASUREMENT BOUNDARIES

Target	Value	Measurement boundary / conditions
Sensor-to-client latency	$<$100 ms	Sensor capture to visible client update, using ESP32 and named client hardware with documented clock offsets
Web3D rendering	60 frames/s	Steady-state rendering in a defined reference scenario on named hardware
State consistency	No unresolved revisions	Across the MR and Web3D clients under specified load and reconnect conditions
Sensing accuracy	Measured against reference	Flow measurement and pulse-count comparison at target operating rates

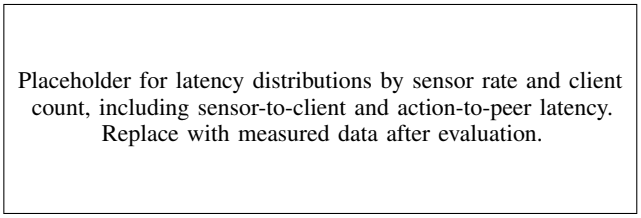


Fig. 4. Bidirectional synchronization latency: (a) sensor-to-client latency across sensor rates and client counts; (b) action-to-peer latency under the same scenarios. To be populated after bench evaluation.

project-coordinate reference fixture where the comparison is valid.

D. Web3D Rendering

The rendering evaluation should report frame-rate distributions on named reference hardware, asset transfer size and load time for compressed and uncompressed mesh variants, and response times for selection and configuration edits. Baseline rendering (without PBR, exploded views, or audio) should be distinguished from enriched configurations so that additional visual complexity is not conflated with the performance of the base system.

E. Interaction and Usability

If the user study receives approval and is conducted, the report should include task-completion rates, task times, selection misses, edit errors, assistance counts, SUS and NASA-TLX scores for each condition, and anonymized qualitative themes. Participant count, relevant characteristics, and previous AR/MR experience should be reported. If only one MR configuration is feasible, the results should be framed as a feasibility study rather than a cross-device comparison.

VII. DISCUSSION

The main synchronization question is whether sensor-owned and user-owned fields converge to a single authoritative revision across the MR and Web3D clients, both during normal operation and after reconnect. The ownership rules are designed to prevent sensor measurements from replacing configuration values and vice versa. They do not, on their own, prove reliable convergence: that claim depends on the implementation of sequencing, acknowledgements, stale-revision handling, snapshot delivery, and event replay. These behaviors must be assessed using the timestamped logs and divergence records described in the evaluation plan.

The 100 ms telemetry target should be interpreted in relation to its declared boundary, from sensor capture to a visible client update. If measurements exceed the target, the recorded timestamps can help identify whether the dominant delay arises during acquisition, transport, server processing, event broadcast, or rendering. Reporting only an average would hide tail delays, so median and tail percentiles should be presented with the corresponding network and load conditions.

Spatial interaction presents a different constraint. HoloLens 2 and XREAL One Pro do not offer equivalent built-in tracking capabilities. A task requiring stable, room-locked 6DoF placement is a reasonable candidate for the HoloLens 2 pathway, whereas the XREAL One Pro pathway may require host tracking or a more limited screen-anchored workflow. This difference should be reported as a portability boundary, not treated as a feature-parity result. Comparisons should account for interaction method, setup effort, and the actual tracking configuration used.

The 60 FPS Web3D target must likewise be reported against named reference hardware and a defined scene. PBR materials, exploded assemblies, and complex meshes may improve visual presentation while increasing rendering cost. Draco compression may reduce asset transfer size, but its effect on load time and rendering should be measured rather than assumed. Baseline results and enriched configurations should therefore be reported separately.

A key design choice is to keep the shared model independent of device-specific presentation. Stable identifiers, graph structure, units, the event envelope, and API semantics are intended to remain reusable across future viewers. Tracking, anchors, input, and rendering are documented per client pathway. This boundary should make extensions more manageable, but its practical value remains to be tested through implementation and cross-client evaluation.

Deployment decisions should follow measured requirements. A single FastAPI process is appropriate for the initial vertical slice; a separate broker is justified only if observed message rates or fan-out make the existing design inadequate. Similarly, per-device alignment calibration and recovery from tracking loss are operational requirements that should be assessed alongside the synchronization and rendering results.

VIII. Threats to Validity and Limitations

A. Tracking Asymmetry

The built-in 3DoF tracking mode of XREAL One Pro does not establish the same world-locked 6DoF capability as the HoloLens 2 pathway. Cross-device conclusions must therefore be limited to the documented configuration of each device. A fair comparison requires either a supported host-tracked configuration for XREAL One Pro or a narrower screen-anchored task definition.

B. Unverified Performance Targets

The 100 ms latency and 60 FPS rendering values depend on sensor rate, network conditions, client count, workload, rendering complexity, hardware, and measurement boundaries. They remain targets until benchmarked. Any future claim of meeting them must include the exact boundary, test conditions, and hardware configuration.

C. Sensor Quality

ADC readings, pulse counting, calibration, interrupt handling, and physical installation can introduce bias or missed events. The evaluation must compare readings with appropriate references and document the operating range. Pulse-count loss should be measured and reported instead of assumed to be zero.

D. Scope

The combined plan includes ESP32 telemetry, multiple MR pathways, precision alignment, spatial audio, PBR materials, Draco compression, and a possible user study. This is a broad scope. The work should prioritize a minimal end-to-end path before adding optional features; enhancements depend on successful baseline operation.

E. Prototype Hydraulics

The current simulator produces deterministic demonstration values and does not validate engineering conclusions about real pipe networks. Its outputs should be labeled as visualization or test data until the simulator is replaced or validated against an appropriate hydraulic reference.

F. Platform Changes

SDKs, firmware, runtimes, and supported APIs change over time. The exact versions of ESP-IDF, the OpenXR runtime, Unity, the XREAL SDK, and the browser should be recorded. Vendor documentation should be checked both during implementation and before submission.

G. Unequal Interaction and Small Samples

Hand tracking, gaze, controller or host input, and setup requirements differ across client pathways. Comparisons must make clear whether they represent complete end-to-end device workflows or isolate a particular interaction method. A small convenience sample can support feasibility observations but does not justify population-wide generalization.

H. Network Reliability

The initial WebSocket broadcast is process-local and requires robust sequencing and reconnect behavior before it can support reliability claims. Sequencing, snapshot restoration, and replay must be implemented and tested before any resilience study. Conclusions about reliability should be conditioned on the exact implementation and test conditions.

IX. Conclusion and Future Work

This paper has described SpatialTwin-IoT, a proposed bidirectional MR and Web3D digital-twin framework for smart fluid systems and interactive industrial assemblies. Building on the PipeLab plumbing-network editor, the design defines three tiers: ESP32-based edge sensing, a FastAPI synchronization service, and MR/Web3D clients. It specifies an update model with explicit origin, timestamp, revision, and field-ownership semantics; a coordinate and anchoring model that separates project-to-physical transforms from device tracking space; and an evaluation plan with pre-registered performance targets.

The evaluation is organized around five questions: whether bidirectional updates converge across clients, what latency and delivery behavior the system achieves, which tasks are feasible under the actual tracking and input constraints of the MR devices, how the Web3D client performs on reference hardware, and which deployment constraints affect cross-reality use. The scope of the contribution is deliberately limited. The work does not claim a new hydraulic method, demonstrated industrial suitability, or general superiority of one headset. It proposes a device-independent plumbing-network model, an explicit synchronization design, a reproducible evaluation plan, and a documented boundary between shared and device-specific functionality.

Future work is to complete the end-to-end implementation, run the bench experiments, and replace the figure placeholders with measured results. A feasibility user study may follow once the required ethics approval is in place. Where engineering conclusions are needed, the demonstration simulator should be replaced with or validated against a suitable hydraulic reference. The shared model can then be extended to additional viewers while retaining stable identifiers, the component-and-port graph, units, and the event envelope.

Acknowledgment

The authors thank the Department of Master of Computer Applications, RV College of Engineering®, for laboratory access and academic guidance. They also acknowledge the open-source communities behind React Three Fiber, FastAPI, the OpenXR working group, and the Espressif ESP-IDF project. Hardware configurations and any measured values reported in a final version should reflect the actual devices and toolchain versions used by the team. Readers should verify vendor documentation when reproducing the system.

References

[1] X. Tu, J. Autiosalo, A. Jadid, K. Tammi, and G. Klinker, "A Mixed Reality Interface for a Digital Twin Based Crane," *Applied Sciences*, vol. 11, no. 20, Art. no. 9480, pp. 1–18, Oct. 2021, doi: 10.3390/app11209480.

[2] X. Tu, J. Autiosalo, R. Ala-Laurinaho, C. Yang, P. Salminen, and K. Tammi, "TwinXR: Method for using digital twin descriptions in industrial eXtended reality applications," *Frontiers in Virtual Reality*, vol. 4, Art. no. 1019080, pp. 1–16, Feb. 2023, doi: 10.3389/frvir.2023.1019080.

[3] A. Valladares-Poncela *et al.*, "On-Device Automatic Speech Recognition and Interaction for Low-Resource Languages in Mixed Reality Industrial Metaverse Applications," *IEEE Access*, vol. 13, pp. 45120–45135, Jan. 2025, doi: 10.1109/ACCESS.2025.3564137.

[4] X. Hao *et al.*, "A Method for Building a Mixed-Reality Digital Twin of a Roadheader Monitoring System," *Applied Sciences*, vol. 14, no. 24, Art. no. 11582, pp. 1–20, Dec. 2024, doi: 10.3390/app142411582.

[5] Y. Lee and B. Yoo, "XR collaboration beyond virtual reality: Work in the real world," *Journal of Computational Design and Engineering*, vol. 8, no. 2, pp. 756–772, Apr. 2021, doi: 10.1093/jcde/qwab012.

[6] S. Vemuri *et al.*, "Cloud-Based Collaborative Virtual Reality Labs and Web-Simulated Microarchitectures," *Journal of Web and Ubiquitous Applications*, vol. 17, no. 1, pp. 10–25, Jan. 2026, doi: 10.1007/s11042-025-20101-z.

[7] T. Kaarlela, P. Padrao, T. Pitkäaho, S. Pieskä, and L. Bobadilla, "Digital Twins Utilizing XR-Technology as Robotic Training Tools," *Machines*, vol. 11, no. 1, Art. no. 13, pp. 1–19, Jan. 2023, doi: 10.3390/machines11010013.

[8] K. Q. Tran, J. D. Velazco-Garcia, and N. V. Tsekos, "A Real-Time Immersive Augmented Reality Interface for Large-Scale USD-Based Digital Twins," *Applied System Innovation*, vol. 4, no. 4, Art. no. 50, pp. 1–18, Dec. 2025, doi: 10.3390/asi4040050.

[9] G. Yenduri *et al.*, "Latency Analysis of WebSocket and Industrial Protocols in Real-Time Digital Twin Integration," *International Journal of Engineering Trends and Technology*, vol. 73, no. 1, pp. 120–135, Jan. 2025, doi: 10.1016/j.eij.2024.100512.

[10] ISARC Consortium, "Bi-Directional Communication Bridge for State Synchronization between Digital Twin Simulations and Physical Construction Robots," in *Proc. 37th International Symposium on Automation and Robotics in Construction (ISARC)*, Kitakyushu, Japan, 2020, pp. 1481–1488, doi: 10.22260/ISARC2020/0205.

[11] H. Li *et al.*, "A digital twin platform for building performance and fluid monitoring using open web standards," *Building Simulation*, vol. 18, no. 3, pp. 401–418, Mar. 2025, doi: 10.1007/s12273-024-1180-2.

[12] Z. Lei, H. Zhou, W. Hu, and G.-P. Liu, "Web-based digital twin online laboratories: Methodologies and implementation," *Digital Twin*, vol. 2, Art. no. 3, pp. 1–14, Mar. 2022, doi: 10.12688/digitaltwin.17563.2.